Sistemas Operativos

2024 – 2025

Sincronização Simples – exclusão mútua

Tópicos

Fundamentos de sincronização – Problema da esclusão mútua

Semáforos binários

Bibliografia específica:

- *Fundamentos de Sistemas Operativos*; 3ª Ed.; Marques & Guedes
- *Operating Systems Concepts* ; Silberschatz & Galvin

Sincronização

Nesta parte da matéria:

- Conceitos sobre sincronização. Problemas de sincronização: como os identificar. Secção crítica
- Conceito de exclusão mútua. Implementação da exclusão mútua a baixo nível
- Conceito de semáforo (usado como mecanismo genérico para resolver problemas de sincronização)
- Conceito de Mutex
- API Mutex (documento à parte e exemplo)

Nota: o API de semáforos em Unix não é abordado.

A conseguir nesta parte a matéria:

- Saber identificar problemas de sincronização (exclusão mútua)
- Saber resolver a nível abstracto com recurso a semáforos/mutexes
- Saber usar o API de mutexes em Unix

Importante: Este documento é um resumo. É importante ver a bibliografia

Sincronização

Motivos para sincronização

1. Num sistema no qual existam em execução simultânea várias *entidades* (processos e *threads*) que utilizem recursos e dados partilhados entre si exige a coordenação do acesso a esses mesmos recursos ou dados

2. Qualquer sistema que seja composto por mais do que uma entidade activa em que uma das entidades seja dependente de acontecimentos originados por outra(s) dessas entidades exige a coordenação da sua execução em função da execução dessas outras entidades

Exemplos típicos da bibliografia

- Gestores de recursos (memória partilhada), gestão de produtores/consumidores

Exemplos concretos da vida real

- Sistemas gestores de dados, sistemas de utilização simultânea por vários utilizadores (ex.: sistemas de vendas de bilhetes; maquinas ATM)

Sincronização

Situações típicas (exemplos) que envolvem sincronização

Cooperação (a ver apenas em SO2)

- Diversas actividades concorrem para a conclusão de uma aplicação comum

Competição (a ver apenas em SO2)

- Diversos processos competem pela obtenção de um recurso limitado
- A competição deve ser resolvida de forma a que o recurso seja utilizado de forma coerente

Exclusão mútua (faz parte de SO)

- A utilização concorrente de uma zona de dados partilhada pode levar a que os dados fiquem inconsistentes
- A utilização deve ser feita de uma forma exclusiva: apenas uma entidade activa utiliza o recurso.
- A execução de uma secção de código que manipula dados partilhados constitui uma situação típica de acesso em exclusão mútua

Sincronização

Plano do estudo da matéria sobre sincronização

1º Exclusão mútua – SO & SO2
- Pode ser vista como um caso particular de competição:
  Competição pelo acesso exclusivo secção de código (secção crítica)
  O problema que se pretende resolver é normalmente o acesso concorrente a dados

2º Competição (SO2)
- Pode ser vista como uma generalização da exclusão mútua:
  A competição é feita sobre recursos com mais do que uma unidade.
  Pretende simplificar a espera/sinalização de recursos. Não envolve necessariamente secções críticas
- As soluções encontradas para a exclusão mútua irão servir de base para as soluções para a competição

3º Cooperação (SO2)
- Os processos querem sincronizar as suas acções de forma explicita uns com os outros. Em vez de competição para avançar, podem mesmo pedir explicitamente para se suspenderem até receberem uma notificação
- Os mecanismos utilizados para os casos anteriores podem ser utilizados com grande simplicidade para conseguir os objectivos da cooperação

Sincronização – Exclusão mútua

O problema na forma mais simples – A exclusão mútua

- Os problemas de sincronização podem ser decompostos a situações simples de exclusão mútua – Se um processo executa uma acção (normalmente sobre dados partilhados) os restantes estão temporariamente excluídos de o fazer
- Nestas situações o problema reside num conjunto restrito de instruções em que existe uma sequência teste-(decisão)-acção. Do resultado do teste irá depender a execução da acção (a qual é em exclusão mutua)
- Idealmente um processo assinalaria os restantes que já se encontraria a executar a acção (fazendo-os abdicar de executar essa acção). O problema ocorre quando o SO retira a execução ao processo que já decidiu executar a acção antes que ele tenha tido tempo de fazer essa sinalização

Sincronização – Exclusão mútua – Secção crítica

Exemplo

Situação

- Uma função que manipula uma estrutura de dados partilhada (**var_partilhada**).
- O acesso deve ser em exclusão mútua (se um processo escreve, os outros aguardam a conclusão da escrita e eventualmente a própria leitura) para evitar leituras de dados incompletos ou destruição de informação ainda não lida

Solução proposta

- O controlo do acesso aos dados é feito através de uma variável indicadora também partilhada (**livre**) mas não vai funcionar pois o teste da variável indicadora **livre** e a subsequente atribuição do valor lógico não-livre a essa variável pode ser interrompido a meio (e então dois ou mais processos poderiam usar a estrutura de dados partilhada **var_partilhada** em simultâneo sem se aperceberem)
- À falta de garantia de funcionamento acresce a utilização de espera activa que ocupa o processador desnecessariamente em testes repetitivos

Sincronização – Exclusão mútua – Secção crítica

Algoritmo da solução proposta para o exemplo anterior

```
Função Escreve (valor)
Começa
   Enquanto livre é falso
      Mantém-se no ciclo
   // saiu => livre => avança
   Coloca livre a falso
   var_partilhada ← valor
   coloca livre a verdadeiro
Termina
```

Espera activa. Só por si já invalida esta "solução"

Se for interrompido entre o teste e o assinalar na variável livre, a exclusão mútua pode falhar (até mesmo o teste da condição no ciclo é falível)

O problema ocorre quando

Entre descobrir que livre está a 1 (significa pode avançar) e colocar livre a zero (decide avançar e assinala que já não está livre) acontece:

- Ocorre uma comutação de processo
- Outro processo executa
- Esse outro processo vê também a variável livre a 1 e portanto decide avançar
- Ambos os processos já decidiram avançar e não voltam a testar nada
- Ambos os processos vão mexer nos dados e gerar incoerência nestes

O problema está nos dados e não no código. Mas é no código que vai ter que ser encontrada a solução deste problema

Sincronização – Exclusão mútua – Secção crítica

Exemplo (código)

```
int escreve(tipo_dados valor) {
    while (1)
      if (livre==1) break;
    livre=0;
    var_partilhada=valor;
    livre=1
}
```

Enquanto livre é falso mantém-se no ciclo.

Feito desta forma é espera activa

Se ocorrer uma comutação de processos aqui a exclusão mútua pode falhar

→Um outro processo veria a variável de controlo livre ainda a 1 e também decidir avançar para a alteração da variável partilhada

→O problema aconteceu porque a sequência teste-decisão-acção foi divisível

O teste da variável indicadora, a actualização do seu valor e a acção (acesso à estrutura de dados partilhada) é uma **secção crítica**

Sincronização – Exclusão mútua – Secção crítica

Secção crítica

- Conjunto de operações que deve ser efectuado de forma atómica (indivisível), normalmente relacionadas com dados partilhados (directa ou indirectamente)

Exemplo típico: acesso a dados em memória partilhada

- A origem da questão que leva a que uma secção de código seja uma secção crítica tem a ver com os dados que são manipulados e não com as instruções propriamente ditas
- Duas zonas distintas de código com aparência completamente díspares podem estar relacionadas através dos dados que manipulam

Sincronização – Exclusão mútua

Soluções algorítmicas para a exclusão mútua:

- Existem diversas propostas algorítmicas (no código do próprio processo e sem passar pelo SO). Exemplos: Algoritmos de Lamport, algoritmo de Dekker, etc. (mais pormenores na bibliografia)
- Estas soluções não resolvem totalmente o problema e implicam o aumento de complexidade do código a cargo do programador

  Exemplos de problemas que ficam por resolver (dependendo do algoritmo usado):
  - Alternância estrita (dois processos acedem alternadamente, limitando-se um ao outro desnecessariamente em muitos casos)
  - Espera activa (desperdício intensivo do processador)
  - Dependência do número de processos envolvidos (soluções não genéricas)

Sincronização – Exclusão mútua – Semáforos

Uma solução para garantir o acesso em exclusão mutua a uma secção crítica

- Impedir a hipótese de ocorrer comutação de processos a meio do teste-decisão-acção (que poderia levar a que outros processo decidisse a mesma coisa por a variável de teste não estar ainda actualizada)

Como:

- Reduz-se o teste-(decisão)-acção a uma única instrução do tipo test-and-set
  Assim já não poderia ser interrompida e p9ortanto não haver comutaçãode processos (eventualmente para outro que fosse mexer nesses dados)
  Instruções Assembly para esse fim
  - Motorola – Instrução TASM – Teste coloca a 1
  - Intel – Instrução XCHG – Troca dois valores entre si

Sincronização – Exclusão mútua – Semáforos

Utilização da instrução XCHG para efectuar um teste-decisão-acção de forma indivisível para controlar o acesso a uma secção crítica

Linguagem de alto nível

```
while (ocupado == 1);        /* 1 significa fechado. Outro processo põe a 0  */
teste = 1;                   /* põe logo a 1 = ocupado */
/* acede a dados partilhados */
```

Compilado (não optimizado)

```
        MOV AX, 1
ciclo:  LOCK XCHG DS:[ocupado], AX  /* LOCK garante bom funcionamento em SMP */
        JNZ endereço                /* ainda tem espera activa, mas ao menos funciona */
        /* acede a dados partilhados */
```

Problema: Ainda mantém a espera activa

Sincronização – Exclusão mútua – Semáforos

Solução ideal para garantir o acesso em exclusão mútua a uma secção crítica e dados

- Manter a indivisibilidade do teste-decisão-acção e acrescentar:

Eliminação da espera activa:

- ***O processo deve ficar bloqueado em vez de estar em ciclo de teste***
- Quando puder avançar é acordado e ter a garantia que pode avançar sem testar mais nada

Problema

- O SO é a única entidade com poderes garantidos para bloquear ou acordar qualquer processo
- O SO não sabe que um processo está À espera de poder avançar. Um clico de teste são apenas mais umas instruções como tantas outras

Solução:

- Transformar a tentativa de acesso a uma zona de código (dados) numa espera de um recurso controlado pelo SO (para o SO saber que o processo está à espera e quando pode avançar)
- O SO bloqueará o processo se esse recurso não estiver disponível e desbloqueá-lo-á quanto voltar a estar disponível
- Este mecanismo deve ser muito simples de usar e genérico
- Existem vários recursos para esse efeito. O mais genérico é o **Semáforo**

Sincronização – Exclusão mútua – Semáforos

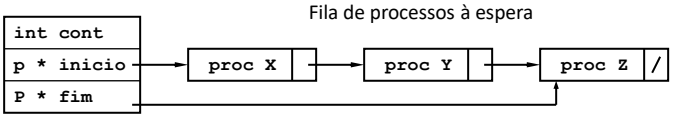
Semáforo

- Recurso controlado pelo SO. Semelhante a um semáforo real, mas com a capacidade de ter um "número de autorizações" (número de processos/threads que o podem ter em simultâneo)
- Constituído por uma estrutura de dados que inclui
  - Variável de controlo (inteira) = número de autorizações restantes
  - Fila de espera (de processos bloqueados)
- As operações sobre semáforos são
  - Esperar – Requisita o recurso / Requisita uma autorização / (pede acesso a uma secção crítica)
  - Assinalar – Liberta o recurso / Devolve uma autorização / (liberta a secção crítica)
- Se o semáforo já não tiver mais nenhuma autorização (contador a zero) o processo que efectua o esperar fica bloqueado. Posteriormente é automaticamente desbloqueado quando algum outro processo efectua esperar / devolve uma autorização. Os "esperar" ficam memorizados e pode haver mais que um processo em espera
- O bloqueio / desbloqueio é ser transparente para o processo

Sincronização – Exclusão mútua – Semáforos

Semáforos

Aspecto de um semáforo



A implementação dos semáforos no núcleo garante que o teste-decisão-acção é de carácter atómico (indivisível), através de instruções *test and set* ou outro mecanismo

Sincronização – Exclusão mútua – Semáforos

Semáforos (em resumo)

- Objectivo: informar o sistema de que o processo está à espera de acesso a um recurso, podendo o SO bloquear a execução do processo até que o recurso esteja livre
- Método: Cada semáforo tem um contador interno. As operações sobre o semáforo são a de **esperar** (diminuir o contador) e **assinalar** (aumentar o contador). Esse contador actua como número de processos que ainda podem passar sem ficarem bloqueados.
- O sistema garante que o teste-decisão-acção interno à implementação do núcleo não é susceptível à ocorrência de comutação de processo, seja por instruções do tipo *test and set*, seja por outros mecanismos disponíveis no núcleo

Sincronização – Exclusão mútua – Semáforos – Esperar

Utilização de semáforos por parte dos processos

Esperar(Semáforo)

- **Pede acesso a uma secção crítica (ou do recurso em questão)**
- Decrementa a variável de controlo
  - Se ficou a zero, então já outro processo está a utilizar a secção crítica guardada pelo semáforo
    - Neste caso o processo é bloqueado, ficando na lista de espera desse semáforo
  - Caso contrário
    - A secção crítica guardada pelo semáforo estava livre
- **Os semáforos para controlo de secções de exclusão mútua são inicializados com a variável de controlo a 1 (apenas um de cada vez pode usar)**

Sincronização – Exclusão mútua – Semáforos – Assinalar

Utilização de semáforos por parte dos processos

Assinalar(Semáforo)

- **Indica libertação de uma secção crítica (ou do recurso em questão)**
- Incrementa a variável de controlo
  - Se houver processos à espera, o primeiro pode avançar
    - O primeiro processo da fila é desbloqueado e o seu estado passa a executável (não é necessariamente posto logo em execução) **(*)**
    - A secção crítica (ou recurso) guardada pelo semáforo fica novamente ocupada
  - Caso contrário
    - A secção crítica (ou recurso) guardada pelo semáforo fica livre

(*) Depende do algoritmo de escalonamento ser ou não preemptivo e da existência de prioridades

Sincronização – Exclusão mútua – Secção crítica

O exemplo anterior agora com semáforos

```
Semáforo sem = CriarSemaforo(1);
/* ... */
int escreve(tipo_dados valor) {
    esperar(sem);
    var_partilhada=valor;
    assinalar(sem);
}
```

Já não há espera activa

O processo bloqueia temporariamente se não poder avançar

O SO garante que apenas um (o número inicial no semáforo) processo estará aqui num dado instante

As funções

- CriarSemaforo (cria um semáforo – o parâmetro é o contador inicial)
- Assinalar (operação *assinalar*)
- Esperar (operação *esperar*)

são simplificações das funções realmente existentes nos SO

Sincronização – Exclusão mútua – Semáforos

- A utilização de semáforos leva ao bloqueio e desbloqueio (transparente para o algoritmo do programa) dos processos que os utilizam

Bloquear um processo envolve

- Retirá-lo de execução
- Salvaguardar o seu contexto (de hardware e de software)
- Marcar o estado do processo como "Bloqueado"
- Colocá-lo no fim da lista de espera

Desbloquear um processo envolve

- Retirá-lo da fila de espera de processos bloqueados onde se encontrava
- Marcar o estado do processo como "Executável"
- Colocá-lo no fim da lista de processos executáveis

  O processo será posto em execução quando o despacho o seleccionar

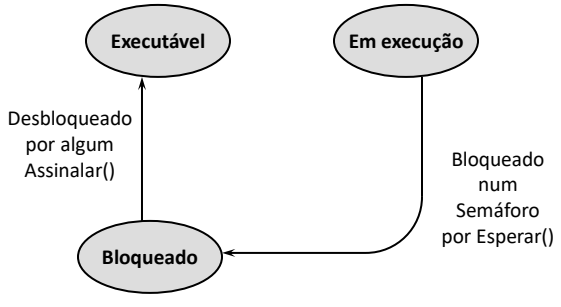
Mecanismos de apoio à sincronização – Algumas notas

A utilização de semáforos:

- **Bloqueia o processo e não ocupa processador**
- O processo pode ficar bloqueado indefinidamente porque o processador pode continuar a executar outros processos
- É um mecanismo justo
  - Existindo uma fila de espera, obtém-se a garantia de atendimento de todos mais cedo ou mais tarde → permite evitar a situação de *starvation*
- É compatível com implementação de políticas de prioridades nos processos bloqueados
- **Elimina a espera activa** – Consiste na principal vantagem sobre os trincos lógicos (trincos lógicos – ver mais pormenores na bibliografia)
- As funções de manipulação dos semáforos devem pertencer ao S.O.
  - As variáveis internas do semáforo não devem ser directamente visíveis
  - Os nomes das funções não são, obviamente, esperar e assinalar, tendo em alguns sistemas um aspecto bastante mais complicado

Sincronização – Semáforos

Semáforos e a transição de estados dos processos



Legenda: Transições que podem ser causadas pela utilização de semáforos

As transições associadas ao escalonamento e os estados "Novo" e "Terminado" não se encontram representados.

Os estados dos processos e *threads* vão ser vistos com mais detalhe mais adiante

Sincronização

Exclusão mútua

- Trata-se na verdade de um caso particular de competição por **um** recurso
  - O recurso é o **acesso** a uma secção de código (na verdade, **dados**)
  - **O número de unidades disponíveis desse recurso é 1**

Pode-se generalizar o conceito de exclusão mútua

- Qualquer recurso pode ser alvo de competição e não apenas secções de código
- O número de unidades disponíveis pode ser qualquer número inteiro maior que zero
- Segundo esta generalização, **os semáforos continuam a poder ser utilizados como mecanismo preferencial de gestão do recurso alvo da competição**
  - **O valor inicial do semáforo é o número inicial de unidades do recurso disponíveis**

Sincronização – Mutex

Mutex

- Resolve directamente o problema da exclusão mútua (**MUT**ual **EX**clusion)
- Geralmente mais simples de usar que um semáforo
- Normalmente disponível nos sistemas *multithreaded*, tal como o Unix
- É abordado no trabalho prático
- **O API e um exemplo encontram-se num documento à parte deste.**

Sincronização – Semáforos Vs. Mutex

Variáveis condicionais

- Abordam o problema de esperar por um acontecimento sem entrar em espera activa
  -> Não é a mesma coisa que a questão de acesso em exclusão mútua
- Normalmente disponível em todos os sistemas operativos.
- Unix: O API de variáveis condicionais está relacionado com o API das threads.
- **O API e exemplo encontram-se num documento à parte deste.**
- Poderá ser útil no trabalho prático

Sincronização – Semáforos Vs. Mutex

Semáforos

- Consiste na generalização do conceito de Mutex, mas em que se permite N processos/threads em simultâneo
  - É útil para resolver situações de coordenação entre processos/threads na gestão de recursos usados por estes
    - Exemplo: produtores/consumidores
  - Se se considerar N = 1, então é como um Mutex
- Normalmente disponível em todos os sistemas operativos.
- Unix: Tem API de semáforos bastante completo, mas não é abordado no trabalho prático e dado apenas a nível conceptual (em SO2 vê-se com detalhe)